Design layer

the whole layer, as one document

Contents

1 Retrace 4

1.1 Foundation 4
1.2 Pending updates 4
1.3 System at a glance 5
1.4 Distribution and verification 6
1.5 End-to-end walkthrough 7

2 Archive 9

2.1 Foundation 9
2.2 Pending updates 10
2.3 Admission 10
2.4 Archival generations 11
2.5 Retrieval 12
2.6 Retention and erasure 13
2.7 Domain model 14
2.7.1 Archive domain 15
2.7.2 Archive generation lifecycle 16
2.8 Collector protocol 16
2.9 Hosting and storage ownership 18

3 Recording 20

3.1 Foundation 20
3.2 Pending updates 21
3.3 Session and recording cursor 21
3.4 Batch and wire format 21
3.5 Domain model 21
3.5.1 Recording domain 22
3.6 Client delivery 23
3.6.1 Typed adapters 24
3.7 Capture coordination 25

4 Replay 27

4.1 Foundation 27
4.2 Pending updates 27
4.3 Replay modules and frames 28
4.4 Viewer 29
4.4.1 Progressive loading 30
4.4.2 Isolated frames for hosted viewers 30
4.5 Build identity and replay artifacts 30
4.6 Example applications 31

Glossary 33

ROOT DESIGN

Retrace

1 Retrace

1.1 Foundation

Retrace records application messages, delivers durable sessions and reconstructs them for inspection.

GOAL

Follow a session across capture, storage and replay

A developer can reproduce an application state from a web or Tauri session, search recordings by user reference, and replay a recording after the collector restarts.

GOAL

Distribute three packages from one Git revision

The portable core, the JavaScript frontend and the BEAM server install from one repository and share one revision.

NO-GOAL

Universal screen recording

Retrace reconstructs explicitly recorded application state. It does not capture pixels, operating-system dialogs, arbitrary DOM mutations or native screens.

NO-GOAL

A hosted deployment policy

Deployment authentication, tenant isolation, retention schedules and consent policy are application and host decisions, and the examples are local development hosts.

INVARIANT

Acknowledgement follows durable commit

An Acknowledgement means PostgreSQL committed the immutable Batch. An object-store failure cannot erase that accepted copy.

partial

PRINCIPLE

Compose small runtime owners

The portable core has no Lustre dependency. A client owns a local delivery queue, the server coordinates PostgreSQL and S3, and the optional Lustre companion composes application-owned replay modules. Importing a package starts no resource.

composition

1.2 Pending updates

ruling **Hosted deployment policy** 2026-10-10

Authentication, tenant isolation, rate limiting, consent and retention schedules need an owner decision before a hosted deployment. The development collector is loopback-only, admits browsers from an origin allow-list and is authenticated only by an optional bearer token.

ruling **Replay beyond a bounded range** 2026-10-10

Live tailing, lazy seeking from checkpoints, event-payload search and analytics are not designed. The viewer retains a bounded range of snapshots.

ruling **Client retry and rejected-batch policy** 2026-10-10

The client retries on fixed delays and a permanently rejected batch blocks its queue. A quarantine and backoff policy needs an owner decision.

ruling **Artifact integrity and registry** 2026-10-11

Artifact digest verification, signing and an atomic manifest registry are recommendations only. The maintained build recipe records provenance and verifies no content.

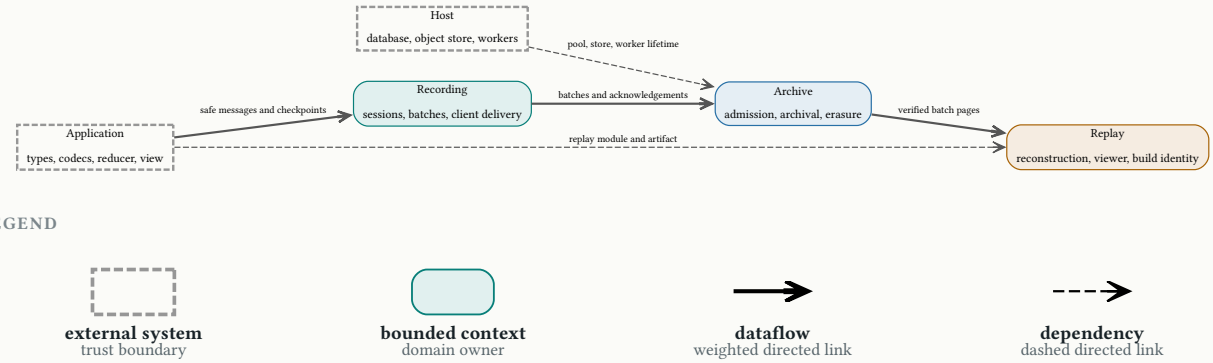
verify **Evidence runs carried over from retired guides** 2026-10-17

The statements on the native desktop check, the Simulator scenario and the physical-phone path describe runs that this review did not repeat. It confirmed only that the retained check files exist.

1.3 System at a glance

Three contexts divide the session path: recording owns the data, archive owns durable storage, replay owns inspection.

VIEWPOINT CONTEXT-OWNERSHIP · ALTITUDE L2 · CONTEXTS Contexts of Retrace



Each context owns its vocabulary. The application and the host supply their own types and resources through explicit values.

Unit	Owns	Caller owns
Portable package	Recording cursor, reconstruction, envelope validation	Types, codecs, clock, deterministic reducer
Frontend client modules	Bounded durable queue, serialized delivery, retries	Capture policy, endpoint, identity, storage selection
Server package	Atomic admission, session index, archival and retrieval	Database pool, object-store configuration, worker lifetime
Frontend Lustre modules	Capture coordination, typed reconstruction, inert frame	Safe codecs, reducer, view, runtime lifetime
Frontend viewer module	Default interface, playback clock, cancellable reads	Source, artifact policy, host mount and cleanup
Example applications	Basic browser adoption and advanced native composition	Starting and stopping local resources

- recording owns the session data model, the wire format, client delivery and capture. It is the upstream context, and archive and replay conform to its terms.
- archive owns admission, indexing, object archival, retrieval and erasure on the server. It supplies verified batch pages to replay through a source the viewer calls.
- replay owns reconstruction, the viewer, replay artifacts and build identity. It never executes application effects.
- Three packages form the distribution boundary: the portable package, the JavaScript frontend that depends on the core and Lustre, and the BEAM server that depends on the core and backend libraries. Frontend delivery, capture, replay and viewer modules keep separate configuration and cleanup, and applications may replace adapters or supply their own viewer or backend. ADR-0008

- The root package has no Lustre, Tauri, database or network import. The examples use PostgreSQL 16, a caller-configured S3-compatible service, Mist on BEAM and a shared Lustre frontend for browser and Tauri, with Nix supplying the development tools. ADR-0001

1.4 Distribution and verification

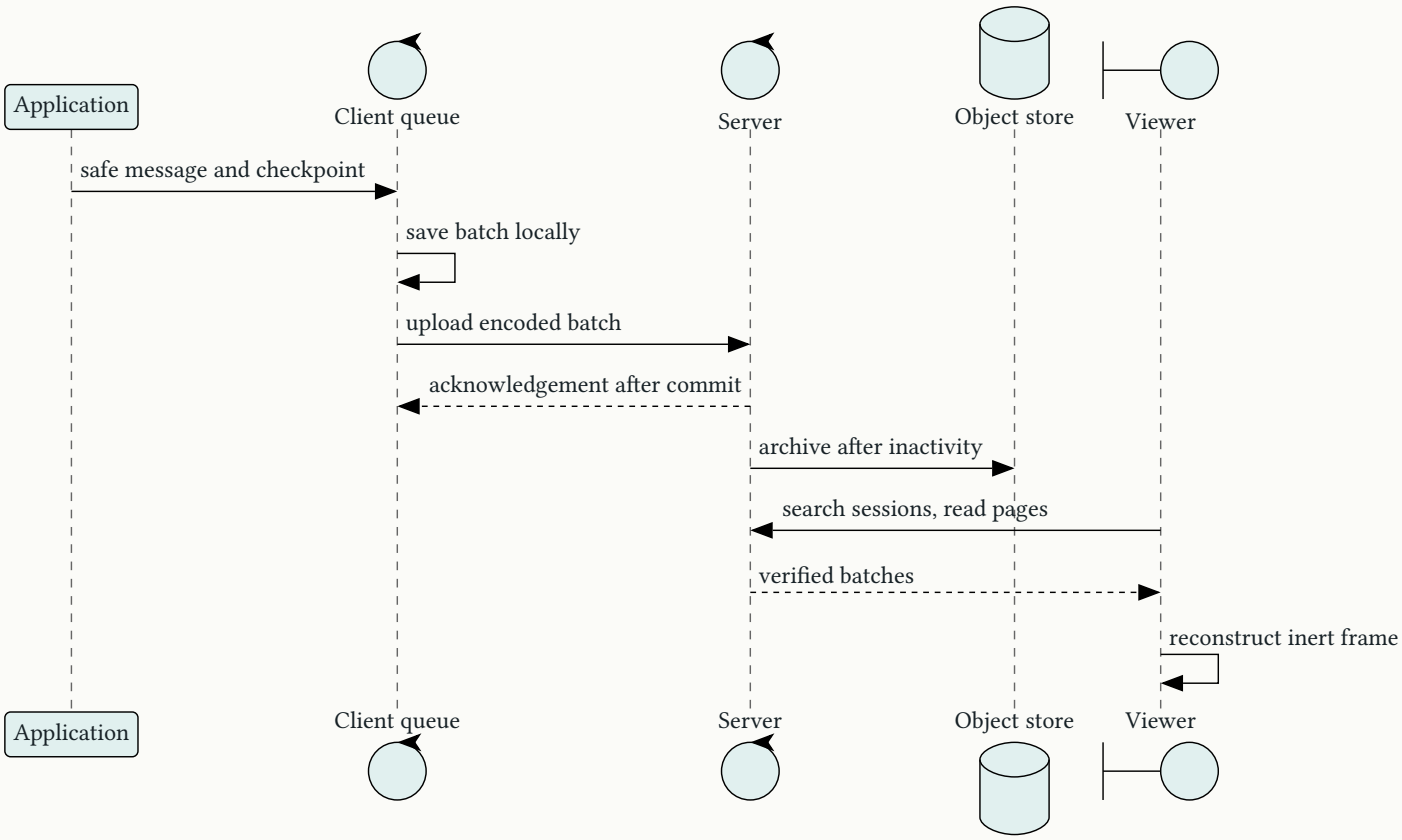
Retrace has three Hex packages, and each kind of evidence is reported separately.

- The release distribution consists of `retrace`, `retrace_frontend` and `retrace_server` on Hex at one coordinated version. An application declares compatible package requirements and commits its `manifest.toml` to retain the resolved versions. The packages are experimental, so an application reviews changes before upgrading. ADR-0020
- An application declares only the packages it imports: the portable core with the frontend for a Lustre browser or Tauri application, and the core with the server for a backend. The core runs on the Erlang and JavaScript targets, the frontend targets JavaScript and the server targets Erlang. Libraries the application imports directly, such as Lustre, remain its own dependencies.
- The repository gate tests both core targets, builds every consumer and both examples, and runs the frontend and artifact tests. A separate Git source consumer gate creates temporary projects outside the repository and runs core reconstruction on both targets, frontend adapter construction on JavaScript and server object-store composition on Erlang. It uses the committed revision, and it exercises no browser, native shell, database or cloud service.
- Usage is shown as compiled, runnable examples, never as a prose guide. Every usage topic has a check that fails when the library's API or behavior moves. ADR-0017 The usage topics list each topic with its source and check.
- The adoption walk-through records an application's accepted transitions, delivers them and replays the delivered batches to the same state. Custom storage, upload and viewer source run against an in-memory stand-in for browser storage.
- The build recipe runs into scratch locations. It shows one build identity in the live and replay bundles, source provenance in the manifest and refusal to overwrite a published build.
- Required validation consists of standard formatting, compilation and behavioral unit and integration tests. Formatting runs through Nix/treefmt, and design validation imports the pinned shared design-layer checker. README snippets are synchronized from compiled example sources by the shared pinned Snips package. Repository conventions and local checker self-tests are not gates. Runnable examples remain tested. ADR-0019
- Native builds, browser tests, database tests and remote-cloud tests are distinct evidence classes, and a compile is no native deployment test. Browser checks keep screenshots in ignored output and create disposable sessions in the development backend. A real PostgreSQL check signs local S3 requests, and an isolated database check injects archive failures and recovery.
- Simulator evidence is separate from browser evidence. The retained Simulator scenario drives the embedded Tauri build with native UI input, uploads through the native queue to the loopback collector, and replays that stored session in a desktop browser. It supports no claim about physical devices, background delivery or power-loss durability.
- A native desktop smoke check drives the real WebView, native store plugin, Rust command and upload with supplied render frames, because a hidden automation WebView pauses animation frames. It is excluded from ordinary builds and says nothing about mobile lifecycle or signing.
- A physical phone reaches a collector only through an endpoint compiled into its frontend, and the loopback default points at the phone itself. The mobile network policy and content security policy must permit that endpoint, and any bridge to the development collector needs an explicit access policy because the collector is open to local callers unless a bearer token is configured. The session ID found and replayed on a desktop browser is the only confirmation that a phone uploaded a session.
- A Hex release comes from a checked source commit with immutable release tags. Before publication, production dependencies use Hex version requirements; published archives are then verified through independent consumers. Git source consumer checks remain separate evidence and do not establish Hex installation. ADR-0020
- Evidence limits stay explicit. The local object store test is no cloud deployment or power-failure test, and a successful build or installation establishes no phone upload. ADR-0012

1.5 End-to-end walkthrough

A recorded transition travels from the application to an archived object and back into a replayed view.

SEQUENCE Record, archive and replay one transition



The client removes its copy only after the acknowledgement. Archival never reduces the committed copy, and replay performs no application effect.

- The application observes an accepted transition and the capture module records its checkpoint and entry as a **Batch** in the **Delivery queue**. The queue saves it locally, uploads it, and removes it when the matching **Acknowledgement** arrives.
- The server commits the batch, archives it after inactivity into an **Archive generation**, and serves it again as verified pages. The viewer selects the **Replay artifact** named by the session's **Build identity** and replays the pages in original time.

CONTEXT

Archive

2 Archive

2.1 Foundation

Archive owns server-side admission, indexing, object archival, retrieval and erasure of recorded sessions.

GOAL

Commit every accepted batch before acknowledging it

The server admits a Batch in a database transaction that locks its Admission cursor, and returns its Acknowledgement only after the commit.

GOAL

Move accepted recordings to object storage without a loss window

The database copy of a batch stays readable until its archive object is uploaded and verified, so an object-store outage only delays archival.

GOAL

Erase a session on caller authority

A caller-authorized Deletion fence stops admission, archival, search and reads for a session, and the caller drives deletion of its objects and recording rows.

NO-GOAL

A hardened hosted service

Authentication, tenant isolation, rate limits, archive repair and a versioned migration system belong to the deployment. The development collector binds to loopback and is not a service configuration.

INVARIANT

A committed batch outlives every archival failure

A failed upload, failed verification or dead worker leaves the database copy readable and eligible for retry.

mechanism

INVARIANT

Admission is ordered per session

A batch is admitted only when its checkpoint matches the session's Admission cursor and its fixed session metadata agrees with the stored metadata.

mechanism

INVARIANT

Original batch identity survives archival

A Batch receipt retains the original range and digest after the database payload is removed, so an exact retry returns the original acknowledgement.

mechanism

INVARIANT

A fenced session never becomes readable again

Admission, archival, search and reads refuse a fenced session, including a late object write that finishes after its worker died.

mechanism

INVARIANT

Erasure never completes while an admitted upload might publish

An Write attempt is committed under the session lock before its PUT is dispatched. Erasure reports completion, clears recording rows and marks the tombstone reconciled only when every attempt is completed or proven quiescent and every inventoried key is deleted. A timeout, expired lease, dead worker, absent object, successful DELETE or local cancellation never proves quiescence.

mechanism

PRINCIPLE

The host owns operations and policy

The host owns the database pool, object-store configuration, worker lifetimes, retention deadlines, authorization and backup restoration. The package starts no hidden worker.

composition

2.2 Pending updates

verify **Publication, fence and recovery statements carried over from retired guides** 2026-10-17

The rules for the publication lock, the fence snapshot, plan recovery and late-object reconciliation come from retired guides. This review confirmed the numeric limits (1000 entries, 32 MiB, sixteen concurrent reads, eight keys per pass) and the generation states. Server checks now exercise attempt admission, fencing and resolution against a controlled provider (ADR 0019); plan recovery after a crash and the behavior of a real provider remain unverified.

2.3 Admission

The server serializes admission per session and distinguishes duplicates from conflicts.

BEHAVIOR

Duplicates are distinguished from conflicts

Given a batch was durably accepted

When a caller retries its range

Then identical content returns the same acknowledgement and different content is refused

level: boundary area: Admission

BEHAVIOR

Session admission is ordered and atomic

Given a session with an Admission cursor

When a new batch is admitted

Then the batch is refused unless its checkpoint matches that cursor and its fixed session metadata agrees with the stored metadata

Then an admitted batch and the session index advance together

level: boundary area: Admission

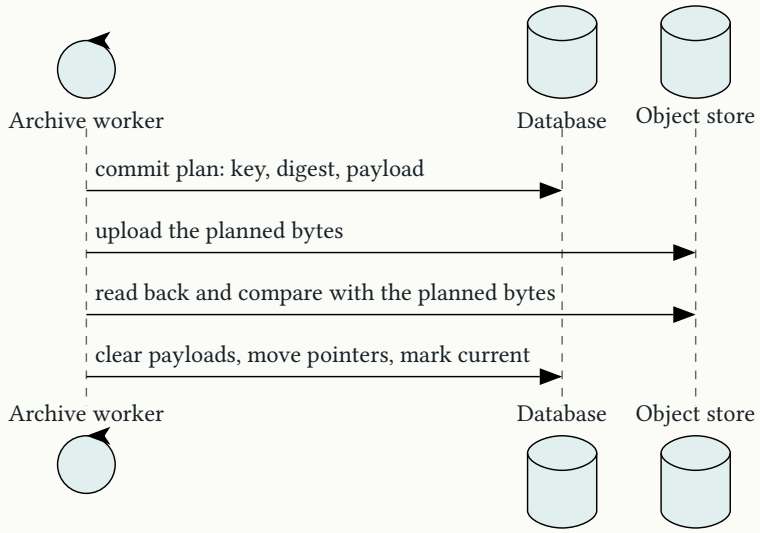
- Admission locks the session row inside a transaction. The initial checkpoint starts at sequence zero. Out-of-order batches are refused. ADR-0001
- A previously accepted batch is identified by session and starting sequence. Its digest permits exact retry checks after its database payload is removed. Different chunking is a conflict, never automatic deduplication.
- A host that must authorize or account atomically with admission calls the in-transaction admission and fence operations inside its own database transaction. Those operations neither begin nor commit a transaction, because nesting a transaction would commit the outer one unexpectedly.

- Session search is bounded to the latest 100 sessions and supports an exact user-reference filter. Search pagination is not provided.

2.4 Archival generations

Archival packs whole batches into immutable generations and preserves the old sources until publication is verified.

SEQUENCE Publish one archive generation



A failure at any step leaves the plan and the old sources in place, so a replacement worker resumes from the committed plan.

BEHAVIOR

Archival failure preserves accepted data

Given a committed batch still held in the database

When object upload fails

Then the database copy remains readable and eligible for a later retry

level: boundary area: Archival

BEHAVIOR

Session archives preserve whole batches

Given an idle session with original durable batch receipts

When the host compacts it or appends a late batch

Then at most 1000 entries occupy each current generation

Then a session no larger than that stays in one object when archivable

Then original checkpoints, page order and exact-retry identities remain unchanged

level: boundary area: Archival

BEHAVIOR

Publication has a durable recovery source

Given a persisted archive plan with its payload and fixed object key

When object upload, verification or the publishing worker fails

Then the old sources remain authoritative until verified publication commits

Then a replacement worker can resume the plan

Then a deletion fence inventories every possible generation

level: boundary area: Archival

- A Batch receipt retains its original range and digest after archival. The per-batch archive operation stores one batch per object. The session operation groups whole original batches greedily into Archive segments of at most 1000 entries after host-configured inactivity, ordinarily 60 seconds.
- A session at or below 1000 entries has one current object when archivable. Only entry count creates another segment, and checkpoints and retry receipts are never rechunked. A late batch replaces its preceding segment when the whole batch fits.
- An Archive generation moves from planned to current to retired and never becomes current again. An Archive plan commits its key, digest and complete payload before any remote PUT. Publication verifies a GET against the exact planned bytes before clearing database payloads and changing pointers.
- A decoded segment is limited to 32 MiB, and decompression stops at that limit. A larger group keeps its existing sources and receives a durable deferral until the session advances, because byte size never creates an early segment. At most one planned payload per session temporarily duplicates up to 32 MiB in the database, which hosts count in capacity policy.
- Preparation snapshots sources under a short session lock, reads objects without that lock, and rechecks the original source pointers and complete segment coverage before committing a plan. Per-batch reads use windows of at most sixteen one-MiB objects under a running byte budget. Admission beyond the snapshot range stays safe.
- Publication admits a Write attempt under the session lock, commits it, then uploads and verifies outside the lock and replaces pointers only after a locked recheck of the fence and the plan. A crash or uncertain PUT leaves a durable plan for retry, an inventoried key and an attempt that blocks erasure completion until the host proves it quiescent. Adapters still bound their external I/O. The archive operation reports true for publication or durable deferral and false for no work or a snapshot invalidated by a competing worker.
- The segment object is gzip-compressed newline-delimited JSON, a versioned session and range header followed by the unchanged encoded batches. Retired keys are eligible for immediate cleanup in passes of at most eight. The reclaim operation rechecks that neither batches nor an active plan reference a key, and later deletion reconciliation follows the host interval.
- Old per-batch objects stay readable and become eligible for idle compaction. A host that must avoid temporary per-batch objects does not run the per-batch archive worker. ADR-0009

2.5 Retrieval

Archived sessions stay replayable, and reads return verified batches in recording order.

BEHAVIOR

Archived sessions remain replayable

Given a batch was successfully archived

When the collector restarts and the viewer requests that session

Then the session remains searchable

Then its recording is fetched and checked before playback

level: boundary area: Retrieval

BEHAVIOR

Concurrent retrieval preserves ordered evidence

Given a caller opts into concurrent reads from an adapter safe for independent read calls

When a bounded archived range is retrieved

Then concurrency stays within the requested bounded budget

Then results remain in recording order

Then every payload is verified

Then any failed object refuses the whole range

level: boundary area: Retrieval

BEHAVIOR

Read workers remain scoped to their caller

Given a concurrent range read has outstanding adapter calls

When the read completes, fails or loses its caller

Then its read workers terminate without stopping borrowed resources

Then a deletion fence observed before return refuses the range

level: boundary area: Retrieval

- Range retrieval is ordered and bounded to 128 batches, and the last returned entry's absolute sequence is the next-page cursor. Serial reads run in the calling process.
- Opt-in concurrent reads clamp their worker budget to one through sixteen and require an adapter safe for independent calls. The budget applies to each read and not to all callers. ADR-0016 Workers belong only to the synchronous read, and adapter operations bound their own external I/O.
- A page fails as a whole when an object is missing, corrupt or unavailable. Each page fetches a shared object once and verifies both its envelope and original batch identities. The object adapter returns compressed bytes, and reconstruction verifies the exact stored content digest after bounded decompression.
- A missing object permits one authoritative pointer refresh only when a concurrent replacement changed its source. Corrupt content never falls back. A final fence check refuses a read fenced while its objects were being fetched.
- A concurrent read owns a temporary coordinator and its read workers, and it waits for them to terminate on success or failure. Caller death also terminates them. An adapter exception is returned as an unavailable-storage failure, and cancellation never stops a provider operation or a resource the adapter created.
- Scheduling uses linked processes, typed subjects and selectors from the standard Gleam Erlang library, and an exception guard around foreign adapter calls guarantees worker cleanup. Native bindings remain for hashing, bounded gzip handling and atomic filesystem storage.

2.6 Retention and erasure

The caller authorizes deletion and drives it through a fence and retained tombstones.

BEHAVIOR

Erasure fences delivery and archival

Given a caller-authorized session deletion

When the caller commits a fence and drives erasure

Then admission and reads refuse the session

Then a fence waits only for upload admission, never for a remote call

Then a fenced session admits no new attempt, so a retained worker cannot admit or retry an upload, while an attempt admitted before the fence may still dispatch and blocks completion until proven quiescent

Then objects are deleted before recording rows

Then retained Tombstone s permit idempotent retry and late-object reconciliation

level: boundary area: Retention

BEHAVIOR

Erasure waits for uploads whose outcome is unknown

Given a fenced session with an admitted upload whose PUT failed or was lost without proof of its outcome

When the caller drives erasure

Then the result reports the count of uploads that might publish and not completion

Then keys of those uploads are not deleted or counted as done, and the tombstone is not marked reconciled

Then completion follows only after the host proves every such upload quiescent and every inventoried key is deleted

level: boundary area: Retention

- A Deletion fence atomically prevents new admission, archive work, search and reads, including for a session ID not yet admitted. It snapshots object keys under the transaction-scoped session lock that admission and upload admission also use. Every planned, current and retired key and every attempt key participates in fencing and repeated erasure.
- Erasure idempotently deletes the objects before clearing recording rows. Each pass handles at most eight keys and reports erasing, awaiting writers with the unresolved count, or erased. A further operation advances the least recently checked unfinished fence or reconciles a completed deletion after a supplied interval, and reports that session and its erasure state.
- A Write attempt is completed when PUT returns success. Otherwise it stays in flight or failed until the host proves it quiescent: erasure takes a host function that Retrace calls on every pass for each such attempt with its state, object key, time since admission and time since a returned failure, and Retrace records the basis of each positive answer. Quiescence is therefore never inferred from elapsed time since admission, because a paused worker can send its PUT arbitrarily later. A host may rely on time since a returned failure only against a provider-qualified maximum request lifetime. Proving an in-flight attempt quiescent needs two facts: that no local dispatch can still happen, and that no already-issued request can publish, and stopping the worker or revoking its credentials establishes at most the first.
- Object removal is supplied separately from the put and get functions of an object store. Tombstone s and object keys are retained because a PUT with an uncertain remote outcome can finish after its worker died, and repeated deletion reconciles that object without making its session readable.
- The host owns authorization, deadlines, worker supervision, retry cadence and backup restoration. It reapplies tombstones after restoring a backup and separately qualifies versioned objects, provider snapshots and physical deletion deadlines. Completed erasure does not establish removal from provider versions or backups.
- Every ingestion and archive worker is upgraded before retention is enabled, because older workers ignore the fence and do not record attempts. Deleted is an additional failure variant of admission and reads. ADR-0015 ADR-0019

2.7 Domain model

Archive owns the session record, the batch receipt, the archive generation and the tombstone.

2.7.1 Archive domain

Session record

aggregate · stateful · archive · owned by archive

The server's record of one session: its fixed metadata and the next sequence it admits.

ATTRIBUTES

- authored **Fixed metadata** · Session metadata
  the identifier, user reference, build, platform and replay contract that every later batch must agree with
- derived **Admission cursor** · Sequence number
  the highest committed sequence, advanced with each admitted batch under the session lock

RELATIONSHIPS

- `1 : n` retains many **Batch receipts**
- `1 : n` is archived in many **Archive generations**

Batch receipt

entity · append-only · archive · owned by archive

The durable record of an admitted batch that survives removal of its database payload.

ATTRIBUTES

- derived **Starting sequence** · Sequence number
  the sequence that identifies the batch within its session
- derived **Range** · Contiguous sequence range
  the original range the batch covered
- derived **Content digest** · Content digest
  the digest that permits an exact retry check after the payload is gone

RELATIONSHIPS

- `n : 1` belongs to one **Session record**
- `n : 0..1` is held by at most one current **Archive generation**

Archive generation

entity · stateful · archive · owned by archive

One immutable archive object for a segment of whole batches.

ATTRIBUTES

- derived **Object key** · Object key
  the fixed key committed with the plan before any upload
- derived **Content digest** · Content digest
  the digest of the planned bytes that verification reads back
- derived **Planned payload** · Archive payload
  the complete payload kept in the database until verified publication
- derived **Generation state** · Generation state generation-state · generation-lifecycle
  planned, current or retired

RELATIONSHIPS

- `n : 1` archives batches of one **Session record**

Tombstone

entity · append-only · archive · owned by archive

The retained record that a session was fenced for erasure.

ATTRIBUTES

- derived **Object key inventory** · Object keys
  every planned, current and retired key of the session
- derived **Erasure completion** · Erased, erasing or awaiting writers
  whether every write attempt is resolved and erasure finished deleting the session's objects and rows
- derived **Write attempts** · Write attempts
  each admitted upload with its immutable key and its in-flight, failed, completed or quiesced state

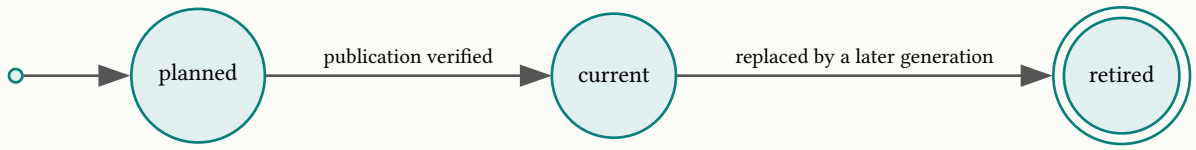
RELATIONSHIPS

- `1 : 0..1` fences one **Session record** when the session was already admitted

2.7.2 Archive generation lifecycle

Generation state planned The key, digest and payload are committed and the object is not yet published. current The object is the verified publication for its entry range. retired A later generation replaced the object and its key awaits cleanup.

STATE MACHINE



2.8 Collector protocol

A collector serves batch submission, session listing and batch paging, and the packaged client and viewer depend on exactly these three operations.

BEHAVIOR

A submitted batch is acknowledged after it is durable

Given a batch within the size limit whose checkpoint matches the session's committed position

When a caller submits the batch

Then the batch is committed before any acknowledgement is sent

Then the acknowledgement names the session and the end position of the batch

level: boundary · area: Collector protocol

BEHAVIOR

Concurrent identical submissions are acknowledged once

Given several callers submit the same accepted batch at the same time

When the submissions race

Then every caller receives the same acknowledgement

Then the session position advances only once

level: boundary · area: Collector protocol

BEHAVIOR

Conflicting and out-of-order submissions are refused

Given a session with a committed position

When a caller submits a batch whose checkpoint does not match that position or whose content differs from an accepted range

Then the submission is refused as conflicting

Then the stored session and its batches are unchanged

level: boundary area: Collector protocol

BEHAVIOR

Invalid and oversized submissions are refused

Given a submission that is larger than one MiB, or whose entries are not contiguous or exceed the entry limit

When a caller submits it

Then an oversized submission is refused as oversized before it is decoded, whether its length is declared or its body is chunked

Then any other invalid submission is refused as invalid

Then no stored state changes

level: boundary area: Collector protocol

BEHAVIOR

Storage failure is reported without internal detail

Given durable storage is unavailable or a stored archive fails verification

When a caller uses any collector operation

Then the caller learns only that recording storage is unavailable

Then storage diagnostics and credentials are not exposed

level: boundary area: Collector protocol

BEHAVIOR

Session listing is filtered, recent and bounded

Given sessions recorded for several user references

When a caller lists sessions with a user reference or without one

Then a listing with a user reference returns only sessions whose user reference equals it exactly

Then a listing without one returns the sessions of every user reference

Then either listing returns at most the 100 most recently updated sessions, newest first

Then each entry reports the session, its committed position, its elapsed recorded time and the counts of batches awaiting archival and already archived

level: boundary area: Collector protocol

BEHAVIOR

Batch paging returns whole batches in recording order

Given a session with accepted batches, some of them archived

When a caller pages batches after a position

Then the page holds at most 128 whole batches whose range ends after that position, in recording order

Then an archived batch is verified before it is returned

Then a page requested after the last accepted position is empty

Then a request without a position starts at the beginning of the session

level: boundary area: Collector protocol

BEHAVIOR

Paging a session with no accepted batches is empty

Given a session identifier with no accepted batches and no erasure fence

When a caller pages its batches

Then the page is empty, as for a known session paged past its end

level: boundary area: Collector protocol

- The three operations are the contract between the packaged client, the viewer and any collector that serves them. A collector may add operations, and it may not change or remove these. ADR-0018
- A collector authorizes callers and scopes tenants itself, because the server package does not. The development collector scopes no tenants, binds to loopback, admits browsers only from an origin allow-list, accepts an optional bearer token and stays outside the hosted-deployment policy. ADR-0021
- Refusals reach callers as distinguishable categories: conflicting, invalid, oversized, erased session and unavailable storage. The client's handling of each category belongs to the separate client retry and rejected-batch ruling.

2.9 Hosting and storage ownership

The server package borrows its resources, and the development collector is the example host.

- The server package borrows a database pool and object-store functions. It starts no HTTP listener, pool or background process on import, and storage functions report explicit read and write failures.
- The package supplies an S3-compatible adapter that takes credentials, bucket and region, and a local directory adapter. A caller may supply its own Object store, so the S3 adapter works with HTTPS and any region and the directory adapter keeps admission unchanged.
- The schema helper creates the initial schema and its archive tables, and the host chooses when to run it before upgraded workers start. It is not a versioned migration system. The schema helper refuses a database that holds tombstones from before write attempts and changes nothing. The runtime role needs select, insert, update and delete privileges on those tables and usage on the attempt sequence.
- The development Collector owns the pool and the periodic archive worker. It serves the collector protocol above and adds local routes for one archive attempt and health. It binds to loopback, refuses non-loopback host names and origins outside its allow-list, requires a bearer token on every route when one is configured, and stores local data in ignored directories. Without a token it logs a warning and stays open to allowed origins and local processes. ADR-0021

CONTEXT

Recording

3 Recording

3.1 Foundation

Recording owns the application-neutral session data and its path from capture to an acknowledged batch.

GOAL

Reconstruct application state from recorded messages

A developer can rebuild the state of an application at any recorded point from the messages and checkpoints the application chose to record.

GOAL

Deliver every saved batch until acknowledged

A client saves a Batch locally before uploading it, retries it after network failure or restart, and removes it only after its Acknowledgement.

NO-GOAL

Automatic redaction

Retrace does not infer which application state is safe to record. Debugger formatters cannot redact data already captured.

INVARIANT

Invalid history never advances a player

A gap, duplicate position, foreign session or contract, backwards time, or failing reducer returns a typed failure and leaves the previous player usable.

mechanism

INVARIANT

Sequence and elapsed time never move backwards

A Recording cursor advances once per Entry. Every sequence and elapsed-time integer fits the JavaScript exact-integer range.

mechanism

INVARIANT

A batch stays queued until its matching acknowledgement

Only an acknowledgement matching the batch session and final sequence removes the batch from the Delivery queue. A late reply from a cancelled upload cannot remove it.

mechanism

PRINCIPLE

The application owns safe meaning

Applications select replay-safe messages, state projections, codecs, user references and Replay contract identifiers before capture. Reducer determinism and purity are application obligations that function types cannot prove.

composition

3.2 Pending updates

verify **Native store and capture statements carried over from retired guides** 2026-10-17

The statements on Tauri native store durability, one writing window per queue and the capture failure paths come from retired guides. This review confirmed the batch limits, the five-second upload deadline, the fixed retry delay and the acknowledgement check, and not the native store behavior.

3.3 Session and recording cursor

Pure values retain application types and report invalid history without advancing a player.

BEHAVIOR

Invalid history preserves the replay cursor

Given a player at a validated checkpoint

When an entry has a gap, duplicate position, different session or contract, backwards time, or a failing application reducer

Then the operation returns the corresponding typed failure and the previous player remains usable

level: boundary area: Reconstruction

BEHAVIOR

A checkpoint can restart reconstruction

Given a compatible checkpoint and subsequent contiguous entries

When the caller reconstructs the recorded range

Then the final state and absolute sequence can be inspected and compared with a supplied checkpoint

level: boundary area: Reconstruction

- A Session has an immutable ID, user reference, build, platform and Replay contract. Multiple devices or tabs use distinct session IDs. The user reference is searchable metadata and never authorization.
- A Recording cursor owns sequence and elapsed-time monotonicity. An Entry advances it once, including a message that changes no state. A Checkpoint starts reconstruction at its declared cursor.
- Start validates identity, record increments the sequence and returns an immutable entry, and resume validates a supplied cursor. The caller recovers its matching outbox and state before resuming.
- Compatibility checks prove neither reducer equivalence nor checkpoint authenticity. The application owns both claims.
- The portable core has no Lustre, Tauri, database or network dependency and runs on the Erlang and JavaScript targets. Application types stay application types, and JSON appears only at the wire boundary.

3.4 Batch and wire format

A batch is the unit of upload, deduplication and replay pagination.

- A Batch holds immutable session metadata, a checkpoint and 1–128 contiguous entries. Its encoded size is at most one MiB. Its checkpoint precedes its entries, and every origin matches its session.
- Application payloads travel as bounded JSON strings, so the collector imports no application types. JSON decoding does not prove the validity of the application's model or event.
- Transport format version and application replay compatibility are separate. Session IDs are unique per recording.
- An application may group up to 128 entries in one batch to reduce request and object counts. It supplies the checkpoint that precedes the group.

3.5 Domain model

Recording owns five application-neutral values: the session, the recording cursor, the entry, the checkpoint and the batch.

3.5.1 Recording domain

Session

entity · immutable · recording · owned by recording

One application's ordered recording, fixed to an application build and replay contract.

ATTRIBUTES

- authored **Identifier** · Session identifier
  the identifier that is unique per recording and independent of the user reference
- authored **User reference** · User reference
  searchable metadata that never authorizes access
- authored **Build identity** · Build identity
  the identity of the application build that recorded the session
- authored **Replay contract** · Replay contract identifier
  the application's compatibility identifier for its schemas and reducer semantics
- authored **Platform** · Platform name
  the platform that ran the application, such as web or Tauri

RELATIONSHIPS

- 1 : n is carried by many **Batches**

Recording cursor

value-object · immutable · recording · owned by recording

The position of a recorder in a session.

ATTRIBUTES

- derived **Next sequence** · Sequence number
  the sequence of the next entry, advanced once per entry
- observed **Latest elapsed time** · Elapsed time
  the latest clock reading, which never moves backwards

Entry

event · immutable · recording · owned by recording

One accepted, safe application message inside a session.

ATTRIBUTES

- derived **Sequence** · Sequence number
  the position the recording cursor assigned
- observed **Elapsed time** · Elapsed time
  the monotonic clock reading when the application accepted the transition
- authored **Application event** · Encoded application event
  the message the application chose to record, encoded as a bounded JSON string

RELATIONSHIPS

- n : 1 belongs to one **Batch**

Checkpoint

value-object · immutable · recording · owned by recording

A replay-safe application model at a known session sequence and elapsed time.

ATTRIBUTES

- derived — **Sequence** · Sequence number
  the session position the model describes
- observed — **Elapsed time** · Elapsed time
  the clock reading at that position
- authored — **Application model** · Encoded application model
  the safe projection of application state that starts reconstruction

RELATIONSHIPS

- `1 : 1` precedes the entries of one **Batch**

Batch

aggregate · immutable · recording · owned by recording

An immutable checkpoint followed by a bounded contiguous range of entries, the unit of upload, deduplication and replay pagination.

ATTRIBUTES

- derived — **Entry range** · Contiguous sequence range
  one to 128 entries whose sequences follow the checkpoint without gaps
- derived — **Encoded size** · Byte count
  the size of the encoded batch, at most one MiB

RELATIONSHIPS

- `n : 1` belongs to one **Session**
- `1 : n` holds many **Entries**

3.6 Client delivery

The client persists before uploading and removes a batch only after its matching acknowledgement.

BEHAVIOR

Interrupted delivery is recoverable

Given a batch confirmed as saved locally

When the network fails or the client runtime restarts before acknowledgement

Then the batch remains available for retry with the same session and sequence

level: boundary · area: Delivery

BEHAVIOR

Capture failure stays visible

Given the local queue is full or its storage write fails

When the application attempts to record another transition

Then capture reports its failure without claiming that transition was saved

level: interface · area: Capture

BEHAVIOR

Typed client adapters preserve operation ownership

Given a client using application-supplied typed storage and upload operations

When an operation completes, fails or its pending upload is cancelled

Then only its first completion is observed

Then cancellation releases the upload and ignores late replies

Then admitted storage work settles and borrowed resources remain caller-owned

level: boundary area: Delivery

- The client reports local save and upload acknowledgement as separate statuses through the application callback. It owns retry timers and exposes shutdown. Retry delays are fixed, and a permanently rejected batch blocks later uploads in its queue.
- A client runtime has one writer and one sequential upload loop. Each page instance starts a fresh session and still drains durable batches left by earlier instances. Duplicate uploads from overlapping runtimes are safe on the server.
- Browser persistence defaults to IndexedDB. Tauri uses an explicitly selected native store adapter, and a browser-only build needs no Tauri plugin. Tauri assumes one writing window per queue and provides no cross-process locking.
- The Delivery queue caps pending capture and durable storage independently by item count and encoded bytes. Both caps together admit up to twice the encoded-payload budget plus metadata, encoding temporaries and storage-engine overhead. The caps are not a process heap limit.
- Admission reserves pending capacity synchronously, and overload stops further capture visibly. Already admitted writes complete unless persistence itself fails. A fresh application session starts after a capture gap.
- The upload deadline of five seconds covers headers and the acknowledgement body. Shutdown aborts the active request and cancels retries. Admitted saves still settle, and shutdown never closes a handle borrowed from custom storage.
- The runtime validates session and final sequence before deleting a batch. Cancellation cannot undo a server commit, so upload retries rely on idempotent admission.
- A live transition and its asynchronous local capture are separate commits, so abrupt termination before the save confirmation can lose that transition. Persisted batches survive a normal runtime restart. Device failure, storage eviction, mobile background execution and power loss are outside this guarantee, and native store durability means only that the plugin save completed.

3.6.1 Typed adapters

- Typed adapter constructors accept cold Task operations and an explicit formatter for application-native errors. Storage and transport policy are independent, and storage and transport may use different error types.
- A Task starts its work when invoked, replies once with a result, and returns a function that cancels pending work. Deferring synchronous work wraps it without running it during adapter construction.
- The bridge observes only the first completion, turns provider throws into failures, and ignores late replies. It starts no scheduler or retry loop and takes no ownership of resources. ADR-0007
- Storage operations must settle, and their cancellation callbacks are never invoked because admitted writes and deletions finish after shutdown. The caller releases storage resources after those operations settle.
- A put must enforce durable item and UTF-8-byte limits atomically, accept identical retries, reject conflicting content, and reply only after saving. Multiple writers are coordinated by the storage implementation.
- Upload receives the exact encoded batch and returns the acknowledgement as encoded JSON, so byte identity is preserved and the application may interpret the batch with the wire module. A custom transport honors cancellation and rejects unsuccessful responses.
- Provider error types stay in application callbacks until the selected formatter produces string diagnostics. Authentication headers never enter recorded events.

Operation	Application supplies
Open store	Opens a store for a queue carrying its name, item cap and byte cap
Store listing	Returns every saved row, each carrying an opaque queue key and an encoded batch body
Store put	Saves one row and replies after saving
Store remove	Removes one row by key
Upload	Sends the encoded batch and returns the acknowledgement JSON
Error formatter	Converts a provider error into a diagnostic string

- Existing opaque JavaScript adapters remain valid. A foreign storage opens a store from a name and two caps and exposes asynchronous listing, put and remove. A foreign transport takes an encoded batch and an abort signal and returns the acknowledgement JSON.

3.7 Capture coordination

The capture module records accepted transitions without taking over application update or effects.

BEHAVIOR

Failed safe projection stops capture

Given an application projects an accepted transition before recording

When the projection fails

Then capture stops visibly without encoding, persisting or uploading that transition

Then private projection errors do not enter capture diagnostics and the live transition remains application-owned

level: boundary area: Capture composition

BEHAVIOR

Capture coordination preserves application ownership

Given an initialized capture state and an application-approved transition

When the application observes that transition with its safe state and event codecs

Then the adapter records its checkpoint and event in order

Then persistence is reported separately

Then a failure stops capture visibly

Then the application's effects are neither executed nor suppressed

level: boundary area: Capture composition

- Capture coordinates the core recorder and client delivery through immutable state and Lustre effects. Starting returns the state and a connection effect, and the application maps capture messages into its own message type. Delivery messages are never observed recursively.
- Observation takes the state before an accepted transition, the message and the safe encoders. It never runs the application reducer or effects. The adapter emits one checkpoint and event batch per observation.
- Capture requires a prior connection. Observing before connection stops capture visibly instead of introducing a silent history gap. Invalid time and failed persistence also stop capture visibly, and the application stays usable.
- The caller owns the client lifetime and stops capture before discarding its state. Retry repairs delivery only and never repairs a failed recording. A fresh session starts after a capture failure.
- Capture information reports the session, captured count, locally saved count, delivery status and optional failure. A per-session monotonic clock and random session ID are supplied by default, and a clock can be injected for deterministic consumers.
- The results of effects, such as a native reply, are recorded as ordinary messages. Time and randomness values that affect state are recorded in events. Only transitions that cannot affect the replay model may be omitted, because an omitted meaningful transition makes later checkpoints inconsistent.
- The opt-in `observe_projected` helper accepts the application's fallible projection of a safe pre-transition checkpoint and safe event. Only successful projections reach the encoders and the ordinary capture path. Projection failure preserves the cursor, stops capture and reports a fixed diagnostic without retaining the application's error. Existing `observe` callers retain their contract.
- Privacy classification and projection belong to the application. Protected fields may retain presence without text, while observations retain the accepted validation, outcomes, loading, locking and presentation decisions needed by the shared view. An annotation cannot establish privacy. No raw recording copy, private sidecar or viewer-only redaction is created.

CONTEXT

Replay

4 Replay

4.1 Foundation

Replay owns reconstruction of recorded ranges, the viewer, replay artifacts and build identity.

GOAL

Inspect a recorded session with the application's own view

A developer can search sessions by user reference, play or seek a bounded recorded range in original time, and see the complete application view with its recorded viewport.

GOAL

Pair every recording with the build that produced it

A Build identity carried by the session selects the preserved Replay artifact that renders it.

NO-GOAL

Native operating-system pixel fidelity

Replay reconstructs explicitly recorded application state. Operating-system sheets, keyboard pixels, arbitrary DOM mutations, scroll positions, theme or locale changes and autonomous animations are not replayed.

INVARIANT

Playback does not execute application effects

The viewer uses an application-supplied pure reducer and an inert view. HTTP, native calls and other effects stay in the live application, and function types alone cannot prove purity.

convention

INVARIANT

A recording replays only against its own build

A missing or incompatible artifact refuses playback with an explanation. The viewer never substitutes the current application build.

mechanism

INVARIANT

An isolated frame accepts only exact authority

An isolated replay frame accepts a message only from the expected window, the exact configured origin, the protocol version and the per-frame nonce.

mechanism

PRINCIPLE

Keep viewer, artifact and application as separate owners

The viewer owns session navigation, playback time and artifact selection. The application owns codecs, reduction, the view and presentation projection, and the host owns storage, authorization and artifact policy.

composition

4.2 Pending updates

verify **Playback and example statements carried over from retired guides** 2026-10-17

The statements on playback timing, viewport scaling and the two example applications come from retired guides. This review confirmed the viewer budgets, the frame channel rules and the build identity recipe, and not playback timing or viewport scaling.

4.3 Replay modules and frames

An optional companion reconstructs bounded ranges using application-native types and a shared inert view.

BEHAVIOR

Protected observations preserve recorded decisions

Given a protected recording whose application codecs validate its observations

When the range is loaded or sought

Then the shared application view receives the recorded decisions without validating placeholder text or invoking live commands

Then malformed observations refuse the range without exposing partially reconstructed snapshots

level: boundary area: Replay composition

BEHAVIOR

Application adapters preserve range identity

Given an application adapter, expected build and a bounded recorded range

When the range is reconstructed from its first checkpoint

Then all envelopes, session metadata, checkpoints and transitions must agree before any snapshots are returned

Then a failure leaves the caller free to retry a corrected range

level: boundary area: Replay composition

BEHAVIOR

Replay frames have an explicit lifetime

Given a mounted application replay frame

When the host disposes it

Then its message listener and application runtime stop responding

level: boundary area: Replay composition

BEHAVIOR

Isolated replay channels require exact authority

Given an explicitly isolated replay frame using a verified application build

When the viewer or frame receives a protocol message

Then it accepts only the expected window, exact configured origin, protocol version and per-frame nonce

Then the frame never infers parent authority from a message, referrer or recording

level: boundary area: Replay composition

- Replay owns no stored entity. `recording` owns the session, batch and checkpoint, `archive` owns the batch receipt, and the application pipeline owns the artifact manifest.
- A `Replay module` is typed by model, message and error. The application supplies its codecs, deterministic reducer, equality, full view, viewport projection and event labels, and the live application and the replay entrypoint share the same view.
- Two strategies use this unchanged contract. Deterministic replay shares a pure reducer and records external results as messages. Protected observation replay records accepted decisions and adopts each validated observation as the next replay state. Both feed the same view used live; neither executes the live effect interpreter.
- Replacing private text before rerunning business validation can change a decision: a one-character input fails a minimum-length rule while a longer protected placeholder passes it. An observation instead records the original invalid decision and field presence. Its decoder validates the observation schema and invariants, never the placeholder as business input.
- The application owns the observation vocabulary, presence policy, codecs, consistency rules and conversion into the shared view's input. Retrace supplies capture, range validation, inert mounting and exact build matching. Excluded text, its

geometry, unrecorded decisions and external state cannot be reconstructed. Function types and inert DOM do not prevent effects hidden inside application callbacks; artifacts must omit live capabilities and use the host's isolation policy.

- Reconstruction validates the complete supplied range before returning snapshots. A range may begin at a nonzero checkpoint, and sequence numbers stay absolute. Every batch checkpoint is verified with the application's equality function.
- Typed reducer errors surface as invalid-replay failures, and decoding or compatibility failures surface as invalid-range failures. A failed call holds no mutable state, so the caller retries with corrected input.
- The `Replay frame` renders the application's full view inertly and ignores its messages. Cleanup removes the frame subscription and the node it owns, is idempotent, and does not rely on the Lustre single-page shutdown branch. One frame runtime serves one iframe document.
- An application with a fixed viewport projects a constant viewport. An application with changing dimensions reads or observes typed viewport values (width, height, top and bottom safe-area insets) and translates them into its own event schema. Presentation fields never enter the portable core.
- The viewport helper observes CSS dimensions and vertical safe areas inside browsers and Tauri WebViews and returns its cleanup. It records no operating-system sheets, keyboard pixels or arbitrary scrolling.

4.4 Viewer

The viewer is the default session-inspection interface with replaceable sources and artifact resolution.

BEHAVIOR

Viewer adapters own cancellable work

Given a viewer using default or application-supplied typed readers

When a request is replaced or the viewer is disposed

Then its pending reads are cancelled

Then stale replies cannot replace the current display

Then disposal can be repeated safely

level: boundary area: Viewer composition

BEHAVIOR

Progressive replay exposes a usable prefix

Given a viewer configured for progressive loading of a bounded recording

When the first valid range is available and later ranges are still loading

Then the developer can inspect the loaded interval

Then subsequent ranges extend the same timeline without moving the playhead

Then a later retrieval failure keeps the valid prefix available with an explicit retry

level: interface area: Viewer composition

BEHAVIOR

Progressive ranges preserve bounded evidence

Given a progressive reader at an accepted checkpoint

When a later page changes session identity, skips an entry, reverses time or exceeds its configured budgets

Then the page is refused without advancing the accepted cursor

Then a corrected retry starts from that cursor

Then cancellation cannot commit late data

level: boundary area: Viewer composition

- The viewer module owns its DOM, scoped styles, playback clock, frame protocol and pending read lifetimes. It imports no Lustre and needs no Tauri plugin or database. Its containing frontend package depends on Lustre.

- A page mounts one viewer, which owns the session and time URL parameters. Mounting refuses an invalid selector and a nonpositive entry budget. The host supplies the element, body margin and surrounding layout, and disposal is idempotent.
- A source supplies session listing by user and ranged reads by id, cursor after and limit. A resolver maps a session to an artifact carrying build, URL and a development flag. Each returns a Task, so selection, repeated search and disposal cancel obsolete work and late replies are ignored.
- A page of batches carries encoded wire strings, and its next cursor is the last sequence when more pages remain. Absolute sequences must advance. A loaded range is validated before display, and a nonzero starting cursor requires its first checkpoint to equal that cursor.
- The default collector reader requests at most 128 batches per read, lists at most the latest 100 sessions, and applies a ten-second deadline. The default artifact resolver checks the manifest api, build and contract. A development override supplies a mutable build and contract without a fallback for other builds.
- The viewer retains at most 5,000 entries by default and refuses beyond its configured budget. Only trusted same-origin artifacts are supported in default mode. Authentication and hiding protected data belong to the host.
- Playback preserves recorded gaps and ends at the last loaded event, and its range caption states that the session end is unknown. Speed changes affect subsequent elapsed time, hiding the viewer pauses it, and a link carrying session and time in milliseconds reopens the same point.
- Loading more recorded events requests the events captured since the previous load. The viewer shows dense markers in clusters and keeps every entry in its event list. A fully different interface reuses the reconstruction, description and mount functions of the companion, because the packaged shell exposes no separate headless playback controller.

4.4.1 Progressive loading

- The JavaScript viewer entrypoint accepts an opt-in progressive configuration with a window duration, window entry count, window byte count, a total byte budget and a page size. The typed mount and existing callers load one complete bounded range. ADR-0013
- A Progressive window keeps whole original batches and targets five minutes, 256 entries or one MiB. One long batch can exceed a time or entry target but never the wire batch limit. An exhausted page is immediately playable and ends its window.
- The selected recording stays within its entry and encoded-byte budgets, and no recording survives selection or disposal. Prefetched data counts toward the same budgets.
- After first playback, the viewer combines the remaining checked windows of one page into a single frame update. It reads at most one page ahead, with one read and one frame update in flight.
- Later windows extend the validated prefix through the existing load and absolute-sequence seek protocol, so historical artifacts need no rebuild. The viewer reports only the acknowledged interval and pauses at its end, and a failed later read keeps the prefix playable and retries the uncommitted cursor.

4.4.2 Isolated frames for hosted viewers

- An explicit opaque frame mode takes a self-contained document from the artifact resolver. The document carries api version, build and the isolated HTML, and the application assembles it from verified immutable bytes.
- The viewer mounts it as a source document with scripts only, never same-origin, form, popup, download or top-navigation capabilities, and suppresses referrers. The assembler embeds scripts, styles and fonts, provides a restrictive content security policy that blocks network access, and injects a fixed trusted parent origin. The origin is never derived from recording content, query parameters, referrers or messages.
- The artifact mounts with the explicit parent origin, which rejects a noncanonical HTTP or HTTPS origin. The viewer places a fresh nonce in the frame name before assigning the document, and the frame accepts messages only from its parent window, that exact origin, the nonce and the protocol version.
- The viewer accepts only the selected frame window, the null origin, the nonce and the protocol version. Outgoing delivery to an opaque frame uses a wildcard target because an opaque origin cannot be named, and no inbound check becomes a wildcard. Default local frames keep same-origin URL behavior. ADR-0014
- Neither the channel nor the sandbox certifies application-supplied reducers as pure.
- A real-browser check loads and seeks the real viewer's script-only frame on a loopback fixture. It verifies that each recording load creates a new named browsing context and disposes it, which the Node protocol tests cannot establish.

4.5 Build identity and replay artifacts

A recording carries its build identity offline, and the application pipeline preserves the artifact that renders it.

Value	Owner	Purpose
Source revision	Git checkout	Full commit SHA of the committed source

Value	Owner	Purpose
Build identity	Application pipeline	Unique identity of one application build and its recording and replay pairing
Replay contract	Application	Compatibility of checkpoints, messages and reducer semantics
Host protocol	Replay integration	Version of commands and responses between the preview host and the artifact
Artifact digest	Application pipeline	Content identity of the completed replay bundle with its styles and assets

- Source revision, build identity and artifact digest stay separate. ADR-0010 One source revision can yield different binaries through configuration, dependency or toolchain changes, so the revision is provenance and never the sole build identity. A bundle cannot embed its own final digest, so the digest is stored in a release manifest outside the bundle.
- The pipeline reads the revision after checkout, allocates one build identity per live and replay pairing before bundling, and writes metadata into ignored build output. No committed file contains its own future commit hash. A dirty flag marks uncertainty and never identifies the changed bytes.
- The live bootstrap and the replay entrypoint receive the same build identity, either injected by the bundler or passed as a startup argument, and the replay entrypoint mounts with the identity written into the session. The application carries its identity offline, so a cached page or an older mobile installation keeps its original identity.
- An application update starts a new session under its new identity. Batches queued earlier keep their original session metadata while they drain and are never rewritten to the uploader's current identity.
- The release order is: build from the intended revision, exercise a recording against its replay artifact, compute the artifact digest, store the immutable bytes, register an immutable manifest from identity to artifact, verify retrieval, then release the already-built application. An identical manifest retry may succeed, different content under one identity fails, and the same bytes are promoted between environments without a rebuild.
- The host distinguishes unavailable artifacts, unsupported host protocols and incompatible recordings and never loads the newest renderer silently. Artifacts are kept as long as their retained recordings need them, including after a rollback. Native platforms may need separate artifacts, and sharing one is a claim to verify.
- Retrace defines no manifest schema, builder package or artifact registry. A multi-file artifact's digest scheme is defined before the digest is treated as portable, and secrets never enter public build metadata.
- Artifacts execute code, so a production host needs a trusted publication path and an explicit isolation and integrity policy. A digest in a filename verifies nothing about fetched content. The advanced example trusts same-origin artifacts and demonstrates style and runtime separation only, and it is no sandbox for untrusted modules.
- The advanced example's build script chooses the build identity from the environment or allocates a unique local one. It injects it into the live bundle and the replay renderer, records source revision and dirty status in the artifact manifest, and writes replay output to a per-build directory. It refuses an existing identity so a later build cannot overwrite a recording's renderer, and it neither hashes, signs nor verifies artifact contents.
- The development server uses a visibly labelled mutable build that preserves no history across source edits. Historical fidelity is tested with an embedded Tauri build or the production bundle. Sessions of an older identity show an unsupported-artifact explanation.

4.6 Example applications

Two maintained applications demonstrate the same session contract with different adapters.

BEHAVIOR

Playback preserves recorded timing

Given a complete validated recording range

When a developer plays or seeks within it

Then the playhead follows elapsed time and preserves gaps

Then the view displays the last state at or before that time

Then equal-time entries retain sequence order and appear together

Then previous and next navigation move between distinct timestamps

level: interface area: Replay

BEHAVIOR

Replay preserves the application viewport

Given a recording with application presentation state

When a developer inspects a point in time

Then the complete application view uses the recorded viewport and screen, scaled to fit without changing its responsive layout

level: interface area: Replay

BEHAVIOR

Unavailable artifacts refuse playback

Given a session whose matching replay artifact is missing or incompatible

When a developer opens it

Then the viewer explains the failure and does not substitute the current application build

level: interface area: Replay

BEHAVIOR

Replay is inert

Given a compatible stored recording

When the viewer navigates its entries

Then the application state is reconstructed without making native requests or recording new live events

level: interface area: Replay

- Exactly two applications are maintained. The basic inventory application uses provided browser storage, HTTP transport, the collector reader and the default viewer, with no handwritten FFI and no native dependency. The advanced membership application adds Tauri, recorded viewport changes, native command results, immutable replay artifacts and an optional custom local archive. ADR-0006
- The basic application's protected topic uses one shared parametric view for private live labels and presence-only observations. Its live-only interpreter fetches a static HTTP result without sending the label; its historical entrypoint imports only the observation schema, shared view and replay runtime. Unit checks inspect saved and uploaded bytes. A separate production-bundle browser check exercises real IndexedDB, seeking, malformed replacement and inert synthetic input with traps for HTTP, storage writes and native commands.
- Each application owns its model, codecs, pure reducer, full view and replay entrypoint, and both use the same public capture, client, reconstruction and viewer APIs. The advanced application changes adapters and forks no shared validation. The shared collector and the backend fault consumer belong to development tooling.
- Each application runs its live entrypoint, viewer and replay frame as separate entrypoints with separate local endpoints. The viewer owns search, artifact loading, timing and event navigation, and the live application owns capture and effect execution. Capture diagnostics stay a separate live-only panel.
- The examples send one event and its checkpoint per batch so each transition is easy to inspect. The advanced example configures a delivery queue of 2,000 batches and 8 MiB and a viewer budget of 2,000 entries, and the basic example keeps the default viewer budget of 5,000 entries.
- A native platform-information request demonstrates a genuine Tauri command whose result is recorded as an application message, and the browser has its own labelled result. Replay displays the stored result without invoking the command again.
- The application records CSS viewport dimensions, safe-area insets and screen navigation in its own model and messages, and the wire format is unchanged. The recorded viewport sizes the replay frame and the viewer scales the frame to fit, so the responsive layout is preserved. The decorative device border is no operating-system simulator.
- The replay document disables animations and user interaction, and adding either requires explicit presentation events and a virtual-time policy. Typography can differ between browser engines.
- An optional custom local archive stores its outbox and accepted archive in browser local storage, acknowledges saved ranges and implements session search and paging in Gleam. It serves one browser writer, needs no collector, and is no production storage recommendation. Only a thin binding to browser I/O and URL selection is handwritten script.

- The default viewer's collector and artifact-directory defaults need no application-authored JavaScript. Applications replace storage reads and artifact policy from Gleam and customize the supplied stylesheet. Presentation of the examples is defined by their stylesheets, and the replay interfaces remain experimental. ADR-0005
- The Replay viewer of the advanced example embeds trusted local artifacts in an iframe through a versioned, source-checked message protocol. It accepts validated checkpoint ranges, bounds retained snapshots, and keeps Time Travel out of its runtime dependencies. The retained attribution records the original derivation. ADR-0004
- The companion and viewer remain experimental, bounded replay without live subscription or an unbounded cache. ADR-0003 The host protocol checks origin, frame identity, a fresh per-load token, protocol version and application build, and those checks do not sandbox hostile artifacts. ADR-0002

Part	Owns	Does not own
Application domain module	Model, safe payloads, reducer, presentation observations	Transport or effects
Application screen module	Complete view reused by live and historical entrypoints	Viewer controls
Replay companion	Build and schema checks, checkpoints, reconstruction, inert frame	Live effects or storage
Application replay module	Codecs, view, labels and viewport projection	Generic replay machinery
Frame entrypoint	Historical Lustre entrypoint and inert view	Native calls or capture
Timeline module	Time-to-state selection and marker grouping	Timers, storage or the DOM
Viewer module	Default interface, frame lifecycle, playback clock	Collector or artifact policy
Viewer host	Collector reads and build-to-artifact resolution	Application model or reducer

Glossary

Every term the layer declares, with the context that owns it.

Acknowledgement

owned by recording

The server's confirmation that a batch is durably committed; it permits the client to remove that batch from its queue.

Avoid: receipt — names the retained record in archive, not the confirmation returned to the client.

Admission cursor

owned by archive

The highest sequence a session has durably committed. The checkpoint of the next admitted batch must equal it.

Avoid: durable cursor — does not say whose position it is; recording cursor — names the recorder's position in recording, not the server's committed position.

Archive generation

owned by archive

One immutable archive object for a segment, passing from planned to current to retired and never returning to current.

Archive plan

owned by archive

The durable record of a generation's object key, digest and complete payload, committed before any remote upload.

Archive segment

owned by archive

A group of whole original batches totalling at most 1000 entries that one archive object holds.

Batch

owned by recording

An immutable checkpoint followed by a bounded contiguous range of entries, used as the unit of upload, deduplication and replay pagination. Archival groups whole batches without changing their receipts or checkpoints.

Batch receipt

owned by archive

The durable record of an admitted batch that retains its session, starting sequence, range and digest after the database payload is removed.

Avoid: acknowledgement — names the confirmation returned to the client, not the retained record.

Build identity

owned by replay

The unique identifier of one application build, written into every session it records and used to select that build's replay artifact.

Avoid: version — implies a release number, while a build identity distinguishes builds of one revision.

Checkpoint

owned by recording

A replay-safe application model at a known session sequence and elapsed time.

Collector

owned by archive

A host that serves batch submission, session listing and batch paging to the packaged client and viewer. The development collector owns a database pool and an archive worker and serves the examples.

Deletion fence

owned by archive

The atomic commit that refuses admission, archival, search and reads for a session and inventories every object key of that session for erasure.

Delivery queue

owned by recording

The client's bounded durable collection of saved batches awaiting acknowledgement.

Entry

owned by recording

One accepted, safe application message with a sequence number and monotonic elapsed time inside a session.

Object store

owned by archive

The caller-supplied pair of put and get functions that place and fetch archive bytes in remote storage.

Progressive window

owned by replay

A group of whole original batches that the progressive viewer publishes to the frame as one update.

Quiescence

owned by archive

The established condition that an upload can no longer publish. A returned PUT success, or a host decision with a stated basis, establishes it; a timeout, expired lease, dead worker, absent object, successful DELETE or cancellation flag does not.

Recording cursor

owned by recording

The position of a recorder in a session, holding the next sequence number and the latest elapsed time.

Replay artifact

owned by replay

A compiled replay module with its document, styles and dependencies, identified by the application build that produced a recording.

Replay contract

owned by recording

The application-owned compatibility identifier for its event schema, checkpoint schema and reducer semantics.

Replay frame

owned by replay

The inert rendering of a replay module's full view inside an iframe document, driven only by loaded batches and seek commands.

Replay module

owned by replay

Application-owned codecs, deterministic reducer and complete view used to reconstruct a recorded application state.

Replay viewer

owned by replay

The session-inspection interface that selects artifacts and controls playback time independently of the live application.

Session

owned by recording

One application's ordered recording, identified independently from its user's reference and fixed to an application build and replay contract.

Task

owned by recording

A cold callback operation that starts its work when invoked, replies once with a result, and returns a function that cancels its pending work.

Tombstone

owned by archive

The retained record that a session was fenced for erasure, held with the session's object keys so that repeated erasure can delete late-written objects without making the session readable.

Write attempt

owned by archive

The durable record, committed under the session lock before an upload is dispatched, of one PUT of an immutable object key. Its state is in flight, failed, completed or quiesced, and only an in-flight or failed attempt might still publish.